CHRIST COLLEGE OF ENGINEERING, IRINJALAKUDA

AI Ignite

Discover the Power of AI Tools

What these tools are, how to use them well, and what is actually inside them.

Abhai Kollara Senior Software Engineer, Zoko

Online · 31 October 2026 · 6:30 to 8:30 PM

INTRO

A short intro

Where I come from

CUSAT, then a research engineer at Saama Technologies, then a contributor to Keras. I have watched this field from the model side and the product side.

I ship this stuff

At Zoko we build AI agents that answer WhatsApp messages for about a thousand businesses. Real customers, real orders, real mistakes.

No hype today

No slides about the future of humanity. Tools you can use tonight, what is inside them, stories from production, and what I would do with the next six months if I were starting now.

WARM-UP · 3 MINUTES

Open the chat.

Which AI tools have you used this week, and for what? One line. Anything counts.

INTRO

The next ninety minutes

10 min	**Explore the tools**	Too many names. Five jobs they actually do
20 min	**Real-world use**	Three live demos from a student’s week
25 min	**What’s inside**	What a model is, what it is not, and a live-coded agent
5 min	**Tips**	Getting better answers, and knowing when to check
10 min	**From industry**	What it looks like when this is your job
10 min	**Questions**	In chat the whole way, out loud at the end

PART 1 · EXPLORE · 10 MINUTES

Too many tools. Five jobs.

Forget the brand names for a minute. Every tool on the poster does one of five things.

PART 1 · EXPLORE

The map, by job

Think and write

Explain, draft, rewrite, brainstorm. The general-purpose chat box.

ChatGPT · Claude · Gemini

Code

Write, fix and explain code. Inside your editor or your terminal.

Claude Code · Cursor · GitHub Copilot

Notes and research

Work over your own documents, or search the web with sources.

NotebookLM · Perplexity · Notion AI

Images and video

Make or edit pictures and clips from a description.

Midjourney · Ideogram · Runway

Inside apps you already use

The same abilities, built into Word, Docs, Gmail, Excel.

Microsoft Copilot · Gemini in Docs · Grammarly

Which of these came up in the chat?

Most of us live in the first column. The other four are less known and often more useful.

PART 1 · EXPLORE

Underneath, they are all the same three things

A model

The part that reads and writes. A handful of companies train them; everyone else rents one.

What it is connected to

Your files, your notes, the web, your editor. This is most of what makes one tool different from another.

What it is allowed to do

Only suggest? Or actually edit files, send messages, run code? This is the knob that turns a chat box into an agent.

Keep these three in mind during the demos. In part 3 we will open one up and see them in code.

PART 1 · EXPLORE

How to pick one: three questions

1

Does it need to see my stuff?

Lecture notes, a codebase, a spreadsheet. If yes, a tool that connects to it beats pasting pieces into a chat box.

2

Does it need to do something, or just tell me?

Explaining a concept is safe anywhere. Editing files, sending emails or running commands is a different kind of tool, ideally one with an undo.

3

Does it have to be right, or just fast?

Brainstorming can be wrong at no cost. A fact in an assignment is a different matter, and tools that show their sources help there.

PART 2 · REAL-WORLD USE · 20 MINUTES

A student’s week, with help.

Three live demos: a bug, a pile of notes, and a chore.

PART 2 · DEMO 1

A bug that gives no error

```
marks = [72, 65, 88, 91]
total = 0
for i in range(1, len(marks)):
    total = total + marks[i]
print("Average:", total / len(marks))

# prints 61.0, but the average is 79.0
```

What to type into the chat tool

> “This prints 61 but the average should be 79. Don’t fix it yet. Explain what is wrong, in one paragraph.”

Then: “Now show me the fix, and tell me how I could have caught this myself.”

Watch for Pasting the whole thing, including what you expected, gets a better answer. Asking for the explanation first makes it easier to spot when the fix is wrong.

PART 2 · DEMO 2

A pile of notes, the night before

01

Upload what you have

Slides, PDFs, your own typed notes, into NotebookLM

→

02

Ask it to quiz you

“Ten questions on this, hardest first. Don’t show answers until I try.”

→

03

Make it point at the source

Every answer cites the page it came from. Click through when it matters.

This is “a model connected to your files”. It only knows what you gave it, which is what you want for exam prep.

Watch for Asking for questions instead of a summary turns reading into practice. It can also explain the ones you got wrong.

PART 2 · DEMO 3

A chore, done by an agent

What to ask Claude Code

> “Write a Python script that sorts my Downloads folder into subfolders by file type. Show me the plan before you write anything.”

Then watch what scrolls past: it reads the folder, writes the file, runs it, reads the error, fixes it, runs it again.

WHAT YOU WILL SEE IT DO

```
Read   Downloads/
Write  sort_downloads.py
Run    python sort_downloads.py
Error  PermissionError: ...
Edit   sort_downloads.py
Run    python sort_downloads.py
Done   37 files moved
```

Watch for It asked before touching your files. This is “a model allowed to act”, and the asking is what makes it usable.

PART 2 · WHAT THE THREE DEMOS HAD IN COMMON

Each one read something, decided, and acted.

The bug fixer read your code. The notebook read your notes. The agent read your folder, then went round that loop a dozen times on its own.

PART 3 · WHAT'S INSIDE · 25 MINUTES

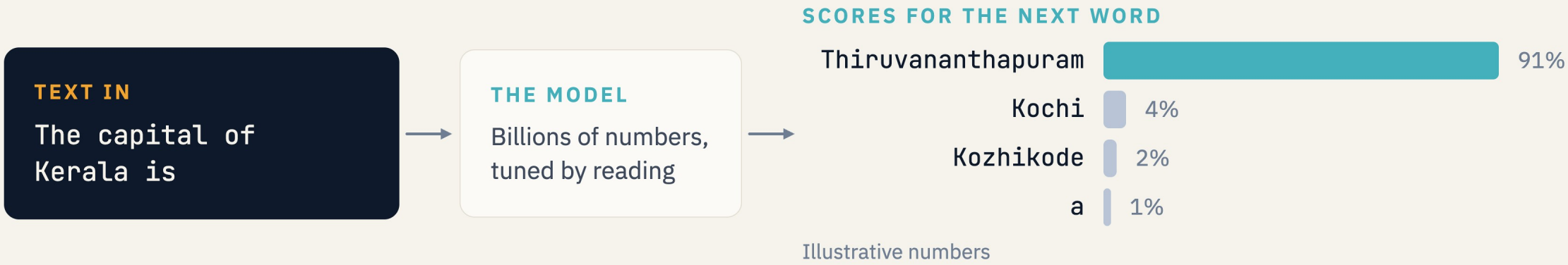
Every tool you just saw is this.

First, what a model is and is not. Then we build the smallest possible agent, live, in three steps.

PART 3 · WHAT'S INSIDE

What a language model actually is

A very large function. You give it some text; it gives back a score for every word that could come next.



It picks one, usually a likely one, sticks it on the end, and does the whole thing again. A paragraph is that step repeated a few hundred times.

PART 3 · WHAT'S INSIDE

How it got that way, and how it runs

Training, once, for months

- Read an enormous slice of the internet, books and code
- Guess the next word, billions of times, and adjust the numbers a little after each miss
- Then get coached by people on which answers are helpful and which are harmful
- Costs more than a building. That is why only a few companies do it

Running, every time you ask

- Your text is cut into tokens, pieces about the size of a short word
- Predict one token. Add it to the end. Predict the next
- Stop when it predicts "end of answer"
- Nothing is looked up. Nothing is saved. Next chat starts from zero

It writes the way you would finish a friend's sentence, except it has read more than any human ever will, and it does it one word at a time, thousands of times, without a plan it can show you.

PART 3 · WHAT'S INSIDE

What it is not

Not a database

It does not look facts up. It reconstructs them from patterns, which is why it can produce a reference that looks perfect and does not exist.

Not a search engine

Unless a tool connects it to one, it has not seen anything written after its training ended.

Not a calculator

It does arithmetic by pattern, like you doing sums in your head. Fine for small numbers, shaky for long ones. Good tools hand maths to real code.

Not a person

No beliefs, no memory between chats, no idea whether its own answer is right. The confidence is a writing style, not a feeling.

Not the same twice

Ask the same question twice and you may get two different answers. That is by design, and it is worth knowing.

Not one fixed thing

"ChatGPT" and "Claude" are products. The model inside is swapped for a better one every few months. What you learn about using them carries over; specific tricks often do not.

PART 3 · WHAT'S INSIDE

So, on its own, it cannot

Remember

Every call starts from zero. The "memory" in a chat app is your app re-sending the history.

See your data

No files, no database, no internet. Unless someone pastes it into the prompt.

Act

It can write a perfect email about a refund. It cannot issue the refund.

Know today

Training stopped at some date. Anything after that is a guess dressed as a fact.

PART 3 · WHAT'S INSIDE

An agent, defined

A model

Reads the context, decides what to do next

+

Tools

Functions you wrote, described so the model can ask for them

+

A loop

Run the tool, feed the result back, repeat until done

=

An agent

Nothing more

Memory, planning, multiple agents talking to each other: all of it is an optimisation on top of these three parts. Here is each part in code.

PART 3 · WHAT'S INSIDE

All the Python we need today

A list

```
files = ["a.txt", "b.pdf"]
files.append("c.png")
```

Things in order. We add to the end.

A dictionary

```
msg = {"role": "user",
       "content": "hi"}
msg["role"]
```

Labels and values. Every message is one.

A function

```
def get_weather(city):
    return data.get(city)
```

Our tool is just a function.

A loop

```
while True:
    ...
    if done:
        break
```

Keep going until we say stop.

If these four look familiar, you can follow the next three slides. If one does not, that is fine: the file will be in the repo and the recording will be online.

PART 3 · LIVE CODING

Step 1: one call to the model

```python
from anthropic import Anthropic

client = Anthropic()          # the library that talks to the model

messages = [{"role": "user",
             "content": "What is the weather like in Thrissur right now?"}]

response = client.messages.create(
    model="claude-haiku-5-5",
    max_tokens=1024,            # the longest reply we allow
    messages=messages,
)
print(response.content[0].text)
```

Run it.

It will explain that it cannot check live weather.

That honest “I can’t” is the reason for step 2.

PART 3 · LIVE CODING

Step 2: give it a tool

```
def get_weather(location):                  # an ordinary function
    data = {"tokyo": "22 C and sunny",
            "thrissur": "32 C and sunny"}
    return data.get(location.lower(), "unavailable")

tools = [{                                  # a dictionary describing it
    "name": "get_weather",
    "description": "Get the current weather conditions for a given city.",
    "input_schema": {
        "type": "object",
        "properties": {"location": {"type": "string"}},
        "required": ["location"],
    },
}]
```

The model never runs code.

It reads the description and replies: “please call get_weather with location = Thrissur”.

Our program decides whether to do it, and does the actual work. Today the “weather” is a dictionary; tomorrow it could be a real weather service. The loop does not care.

PART 3 · LIVE CODING

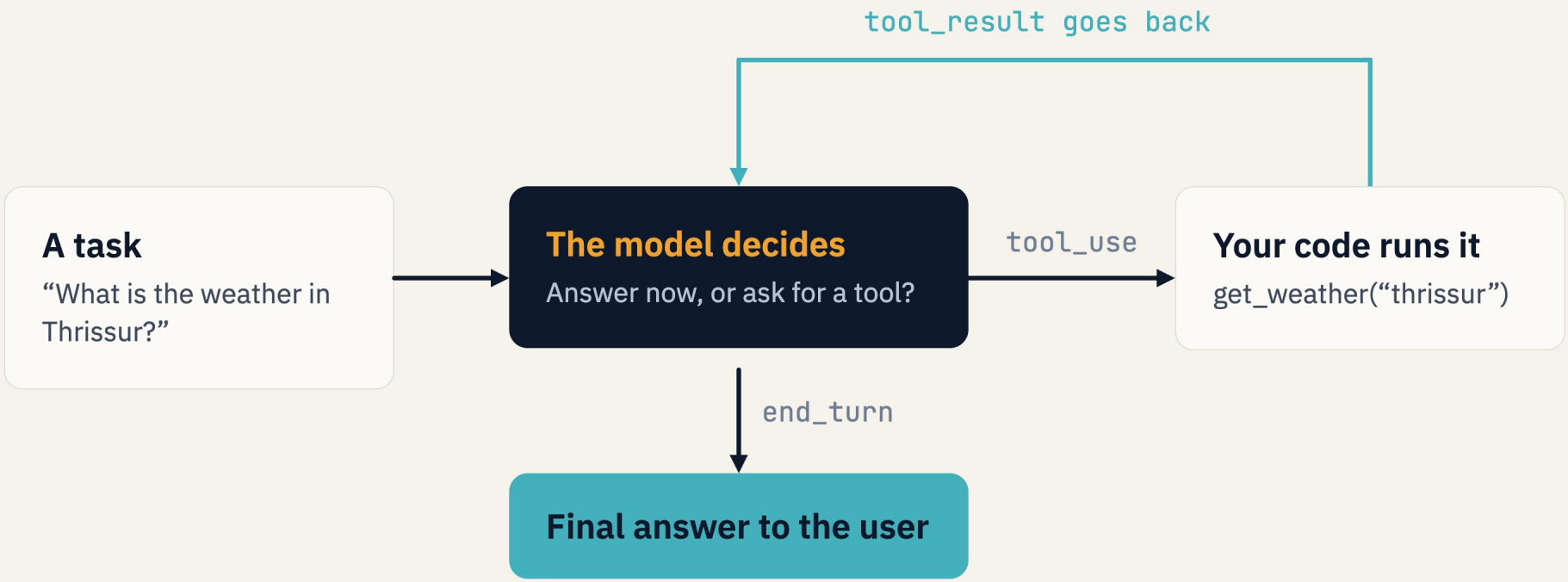
Step 3: the loop

```
while True:
    response = client.messages.create(model="claude-haiku-5-5", max_tokens=1024,
                                      tools=tools, messages=messages)
    messages.append({"role": "assistant", "content": response.content})    # remember what it said
    if response.stop_reason ≠ "tool_use":
        break                           # no tool asked for: it has answered. Stop.
    tool_results = []
    for block in response.content:
        if block.type == "tool_use" and block.name == "get_weather":
            output = get_weather(block.input["location"])        # we run the function
            tool_results.append({"type": "tool_result",
                                 "tool_use_id": block.id, "content": output})
    messages.append({"role": "user", "content": tool_results})    # send the answer back

for block in response.content:
    if block.type == "text": print(block.text)
```

A list, a dictionary, a function, a loop. That is the whole agent. Everything after this is making it better, safer and cheaper.

What just happened



That’s it. That’s every tool on the map.

The bug fixer, the notebook, the agent that sorted a folder, the bots at Zoko: the same loop you just watched get typed. What separates a demo from a product is three things.

Better tools **Instructions that explain the job** **Choosing what the model gets to see**

PART 4 · TIPS · 5 MINUTES

Getting more out of it.

A few things that make the answers better, and a few places where a second look pays off.

PART 4 · TIPS

How to ask

Give it the whole picture

The error, the code, the assignment brief, the constraint. Half the “bad answers” are answers to half a question.

Say what you want back

“A hint, not the solution.” “Three options with trade-offs.” “A plan first.” It will do whichever you ask for.

Ask it to explain its answer

The explanation is usually more useful than the answer, and it costs nothing extra to ask for.

Push back

“Are you sure?” and “what would break this?” work surprisingly well. It is confident by default, not correct by default.

PART 4 · TIPS

Where it tends to slip

Worth a second look

- Facts, dates and numbers
- Citations and references: it will invent plausible ones
- Anything about recent events
- Code you have not run
- Maths with more than a few steps

Usually solid

- Explaining a concept you can test against your textbook
- Rewording, summarising, translating your own text
- Brainstorming, where wrong ideas cost nothing
- Answers that cite a source you can open

A very well-read friend who never says “I’m not sure”.

PART 4 · TIPS

Two ways to ask the same tool

Asking it to teach

- “Give me a hint, not the answer”
- “Here is my wrong answer. Where did I go wrong?”
- “Quiz me on this until I get five in a row”
- “Review my code and tell me what a senior engineer would say”

Asking it to do

- “Do this assignment”
- “Write my lab report”
- Paste the answer, submit, move on

Faster tonight. Nothing left over for the exam.

Both are one prompt away. The difference is a few words.

PART 5 · FROM INDUSTRY · 10 MINUTES

What it looks like when this is your job.

What a normal day looks like, two things that broke, and what to get good at.

PART 5 · FROM INDUSTRY

What I actually use, most days

Mostly Claude Code.

The same loop from slide 23, pointed at a real codebase. A typical day:

- Reading code I did not write, before I touch it
- Fixing a bug with the failing log pasted in
- Writing the tests after the fix
- Boring changes across many files
- Drafting the description of the change for review

Still me

- Deciding what to build and what to leave alone
- Reading every change before it ships
- Talking to the person who hit the bug
- Knowing when the agent is confidently wrong

The tool writes most of the lines. The job is everything around the lines.

PART 5 · FROM INDUSTRY

What to get good at

Matters more than ever

- Understanding what your code actually does
- Debugging: finding out why it did not work
- The fundamentals coming up: data structures, algorithms, how computers work
- Explaining a problem clearly in writing

New, and nobody teaches it

- Describing a task so a model gets it right
- Checking a model's answer
- Deciding what to show it and what to leave out
- Noticing when it is confidently wrong
- Speed and cost as design choices

Matters less

- Memorising syntax
- Knowing every library
- Writing the same boilerplate by hand
- Raw typing speed

PART 5 · FROM INDUSTRY

If I were starting now

1

Get comfortable with the Python basics

Everything today sat on lists, dictionaries, functions and loops. That is enough to build with.

2

Build one small agent from today’s code

Give it one tool that does something you want: check a timetable, tidy a folder, summarise your notes.

3

Show it to five friends

Watch where they get stuck. That is where the next version comes from.

4

Keep a note of the times it gets it wrong

What you asked, what it said, what was off. Professionals keep the same list and call it an eval set.

QUESTIONS · 10 MINUTES

Questions.

In chat, or unmute. Anything goes.

REACH ME

hello@abhai.dev

WHERE I WORK

zoko.io